

Foundations of R Software

Lecture 13

Basics of Calculations

⋮

Matrix Operations – Row, Column & Other Operations

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

Matrix Operations

Renaming the row and column names

```
> x = matrix( nrow=4, ncol=3, data=c(1:12) )
```

```
> x
```

	[, 1]	[, 2]	[, 3]
[1,]	1	5	9
[2,]	2	6	10
[3,]	3	7	11
[4,]	4	8	12

rownames(x) Renames the row names

colnames(x) Renames the column names

Matrix Operations

Renaming the row and column names

```
> rownames(x) = c("r1", "r2", "r3", "r4")
```

```
> x
```

	[,1]	[,2]	[,3]
r1	1	5	9
r2	2	6	10
r3	3	7	11
r4	4	8	12

```
> colnames(x) = c("c1", "c2", "c3")
```

```
> x
```

	c1	c2	c3
r1	1	5	9
r2	2	6	10
r3	3	7	11
r4	4	8	12

Matrix Operations

```
R Console

> x = matrix( nrow=4, ncol=3, data=c(1:12) )
> x
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
> rownames(x) = c("r1", "r2", "r3", "r4")
> x
      [,1] [,2] [,3]
r1      1    5    9
r2      2    6   10
r3      3    7   11
r4      4    8   12
> colnames(x) = c("c1", "c2", "c3")
> x
      c1 c2 c3
r1     1  5  9
r2     2  6 10
r3     3  7 11
r4     4  8 12
> |
```

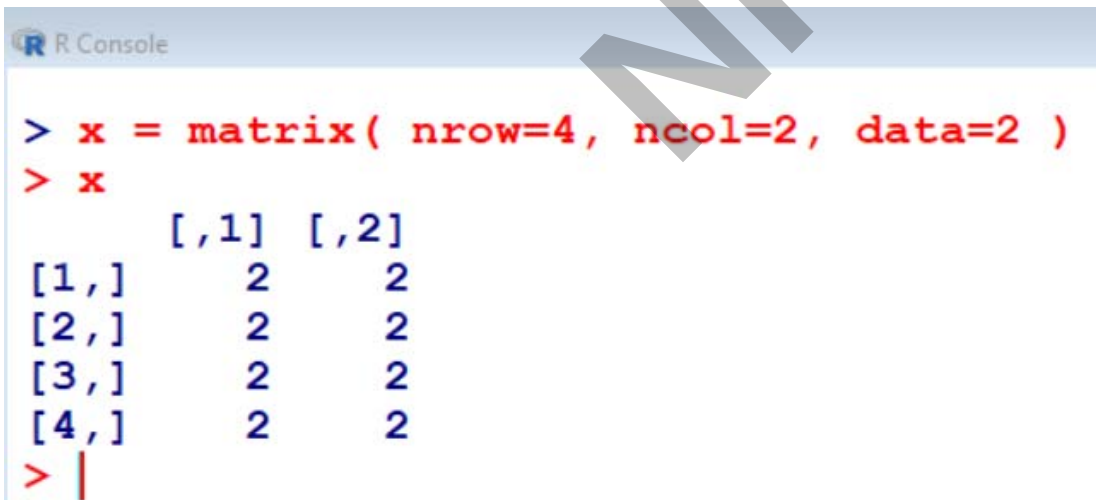
Matrix Operations

Assigning a specified number to all matrix elements:

```
> x = matrix( nrow=4, ncol=2, data=2 )
```

```
> x
```

	[,1]	[,2]
[1,]	2	2
[2,]	2	2
[3,]	2	2
[4,]	2	2



```
R Console  
> x = matrix( nrow=4, ncol=2, data=2 )  
> x  
      [,1] [,2]  
[1,]    2    2  
[2,]    2    2  
[3,]    2    2  
[4,]    2    2  
> |
```

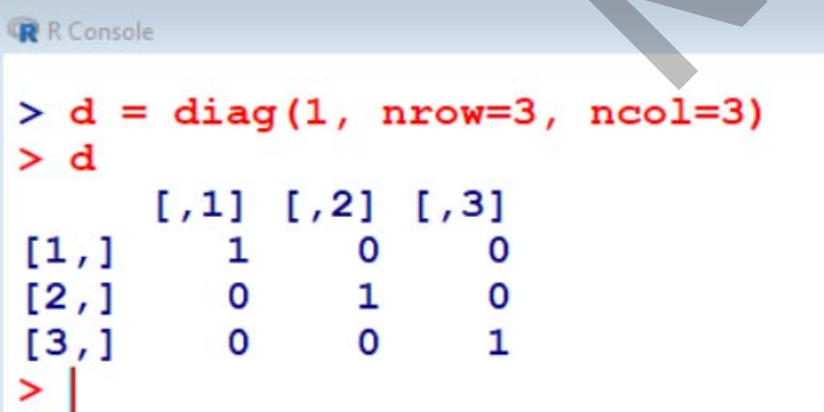
Matrix Operations: Diagonal matrix

Construction of a diagonal matrix, here the identity matrix of a dimension 3:

```
> d = diag(1, nrow=3, ncol=3)
```

```
> d
```

	[,1]	[,2]	[,3]
[1,]	1	0	0
[2,]	0	1	0
[3,]	0	0	1



```
R Console  
> d = diag(1, nrow=3, ncol=3)  
> d  
      [,1] [,2] [,3]  
[1,]    1    0    0  
[2,]    0    1    0  
[3,]    0    0    1  
> |
```

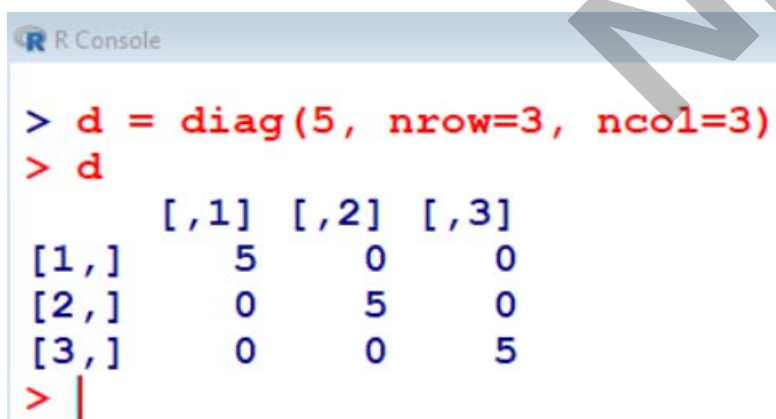
Matrix Operations: Diagonal matrix

Construction of a diagonal matrix with all numbers as 5 of a dimension 3:

```
> d = diag(5, nrow=3, ncol=3)
```

```
> d
```

	[,1]	[,2]	[,3]
[1,]	5	0	0
[2,]	0	5	0
[3,]	0	0	5



```
R Console  
> d = diag(5, nrow=3, ncol=3)  
> d  
      [,1] [,2] [,3]  
[1,]    5    0    0  
[2,]    0    5    0  
[3,]    0    0    5  
> |
```

Matrix Operations: Transpose

Transpose of a matrix X: X'

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T )
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

R Console

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T )
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> |
```

Matrix Operations: Transpose

Transpose of a matrix X: X'

```
> xt = t(x)
```

```
> xt
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	3	5	7
[2,]	2	4	6	8

```
R Console
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T )
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> xt = t(x)
> xt
      [,1] [,2] [,3] [,4]
[1,]    1    3    5    7
[2,]    2    4    6    8
> |
```

Matrix Operations: Row and column sums

Finding the row and column sums

```
> x = matrix(nrow=4, ncol=2, data=c(1,2,3,4,5,6,7,8))
```

```
> x
```

	[, 1]	[, 2]
[1,]	1	5
[2,]	2	6
[3,]	3	7
[4,]	4	8

rowSums(x) Finds the sum of numbers in rows

colSums(x) Finds the sum of numbers in columns

Matrix Operations: Row and column means

```
> x
```

	[,1]	[,2]
[1,]	1	5 → (1+5) = 6
[2,]	2	6 → (2+6) = 8
[3,]	3	7 → (3+7) = 10
[4,]	4	8 → (4+8) = 12

(1+2+3+4) = 10 (5+6+7+8) = 26

```
> rowSums(x)
[1] 6 8 10 12
```

```
> colSums(x)
[1] 10 26
```

```
R Console
> x = matrix(nrow=4, ncol=2, data=c(1,2,3,4,5,6,7,8))
> x
      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
> rowSums(x)
[1] 6 8 10 12
> colSums(x)
[1] 10 26
> |
```

Matrix Operations: Row and column means

Finding the row and column means

```
> x = matrix(nrow=4, ncol=2, data=c(1,2,3,4,5,6,7,8))
```

```
> x
```

	[, 1]	[, 2]
[1,]	1	5
[2,]	2	6
[3,]	3	7
[4,]	4	8

rowMeans(x) Finds the means of rows

colMeans(x) Finds the means of columns

Matrix Operations: Row and column means

```
> x
```

	[,1]	[,2]	
[1,]	1	5	$\rightarrow (1+5)/2 = 3$
[2,]	2	6	$\rightarrow (2+6)/2 = 4$
[3,]	3	7	$\rightarrow (3+7)/2 = 5$
[4,]	4	8	$\rightarrow (4+8)/2 = 6$

$(1+2+3+4)/4 = 2.5$

$(5+6+7+8)/4 = 6.5$

```
> rowMeans(x)
```

```
[1] 3 4 5 6
```

```
> colMeans(x)
```

```
[1] 2.5 6.5
```

```
R Console
> x = matrix( nrow=4, ncol=2, data=c(1,2,3,4,5,6,7,8) )
> x
      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
> rowMeans(x)
[1] 3 4 5 6
> colMeans(x)
[1] 2.5 6.5
> |
```

Foundations of R Software

Lecture 14

Basics of Calculations

::::

Matrix Operations – Access and Mathematical Operations

Shalabh

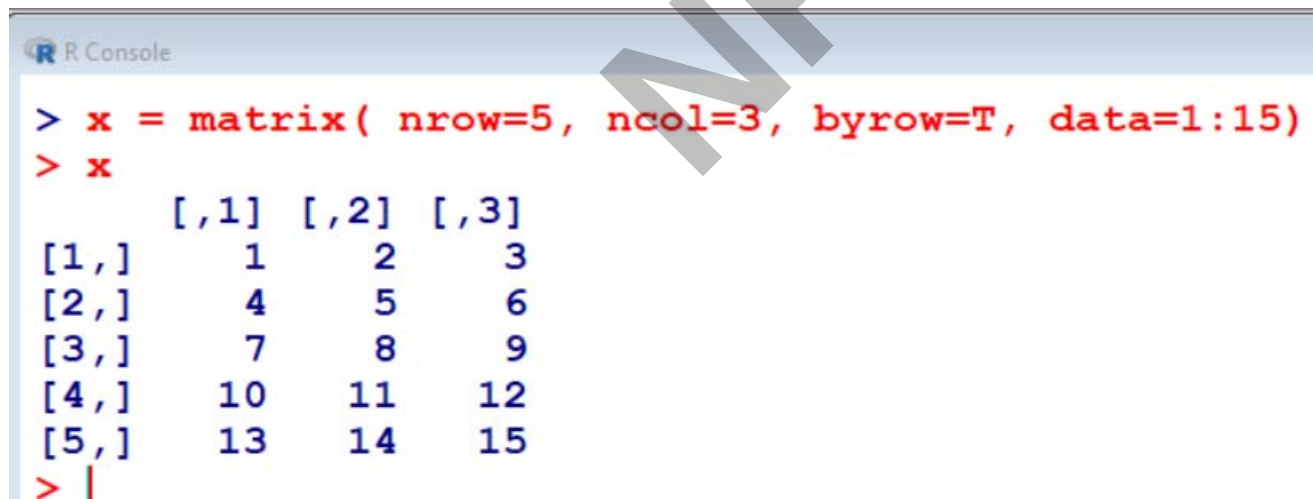
Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

Access to rows, columns or submatrices

```
x = matrix( nrow=5, ncol=3, byrow=T, data=1:15)  
x
```

	[,1]	[,2]	[,3]
[1,]	1	2	3
[2,]	4	5	6
[3,]	7	8	9
[4,]	10	11	12
[5,]	13	14	15



```
R Console  
> x = matrix( nrow=5, ncol=3, byrow=T, data=1:15)  
> x  
      [,1] [,2] [,3]  
[1,]    1    2    3  
[2,]    4    5    6  
[3,]    7    8    9  
[4,]   10   11   12  
[5,]   13   14   15  
> |
```

Access to rows, columns or submatrices

```
> x[3,]  
[1] 7 8 9
```

```
> x[,2]  
[1] 2 5 8 11 14
```

```
> x[4:5, 2:3]  
      [,1] [,2]  
[1,]    11    12  
[2,]    14    15
```

R Console

```
> x[3,]  
[1] 7 8 9
```

```
>
```

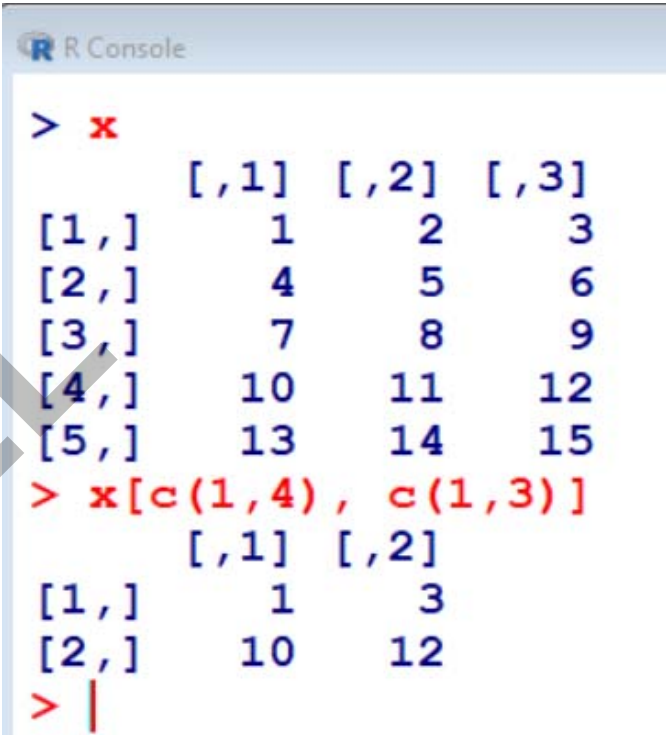
```
> x[,2]  
[1] 2 5 8 11 14
```

```
>
```

```
> x[4:5, 2:3]  
      [,1] [,2]  
[1,]    11    12  
[2,]    14    15
```

Access to rows, columns or submatrices

```
> x[c(1,4), c(1,3)]  
      [,1] [,2]  
[1,]    1    3  
[2,]   10   12
```



The screenshot shows an R console window with the following content:

```
R Console  
> x  
      [,1] [,2] [,3]  
[1,]    1    2    3  
[2,]    4    5    6  
[3,]    7    8    9  
[4,]   10   11   12  
[5,]   13   14   15  
> x[c(1,4), c(1,3)]  
      [,1] [,2]  
[1,]    1    3  
[2,]   10   12  
> |
```

The console output shows the full matrix `x` and then the result of the submatrix extraction `x[c(1,4), c(1,3)]`, which matches the output shown in the first block.

Access to rows, columns or submatrices

```
> x
```

	[, 1]	[, 2]	[, 3]
[1,]	1	2	3
[2,]	4	5	6
[3,]	7	8	9
[4,]	10	11	12
[5,]	13	14	15

Look at the outcome

```
[1] 2 5 8 11 14
```

Not possible to know if It is a row or a column.

Addition of a matrix with a constant

Every element is added with the scalar

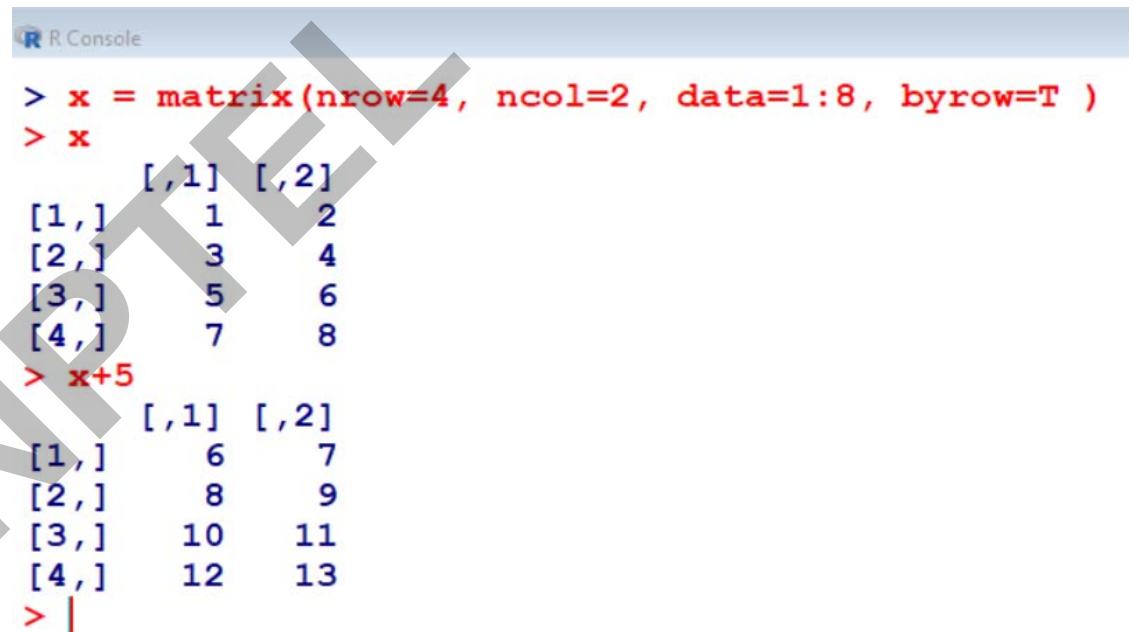
```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> x+5
```

	[,1]	[,2]
[1,]	6	7
[2,]	8	9
[3,]	10	11
[4,]	12	13



```
R Console
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> x+5
      [,1] [,2]
[1,]     6    7
[2,]     8    9
[3,]    10   11
[4,]    12   13
> |
```

Subtraction of a matrix with a constant

Every element is subtracted by the scalar

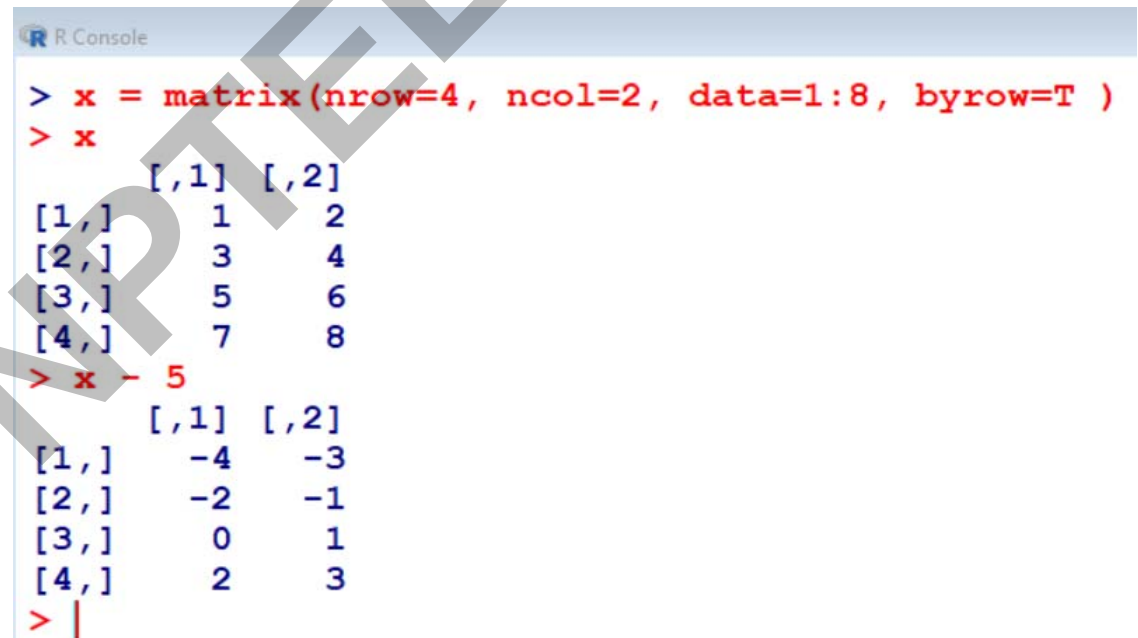
```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> x - 5
```

	[,1]	[,2]
[1,]	-4	-3
[2,]	-2	-1
[3,]	0	1
[4,]	2	3



```
R Console
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> x - 5
      [,1] [,2]
[1,]   -4   -3
[2,]   -2   -1
[3,]    0    1
[4,]    2    3
> |
```

Multiplication of a matrix with a constant

Every element is multiplied by the scalar

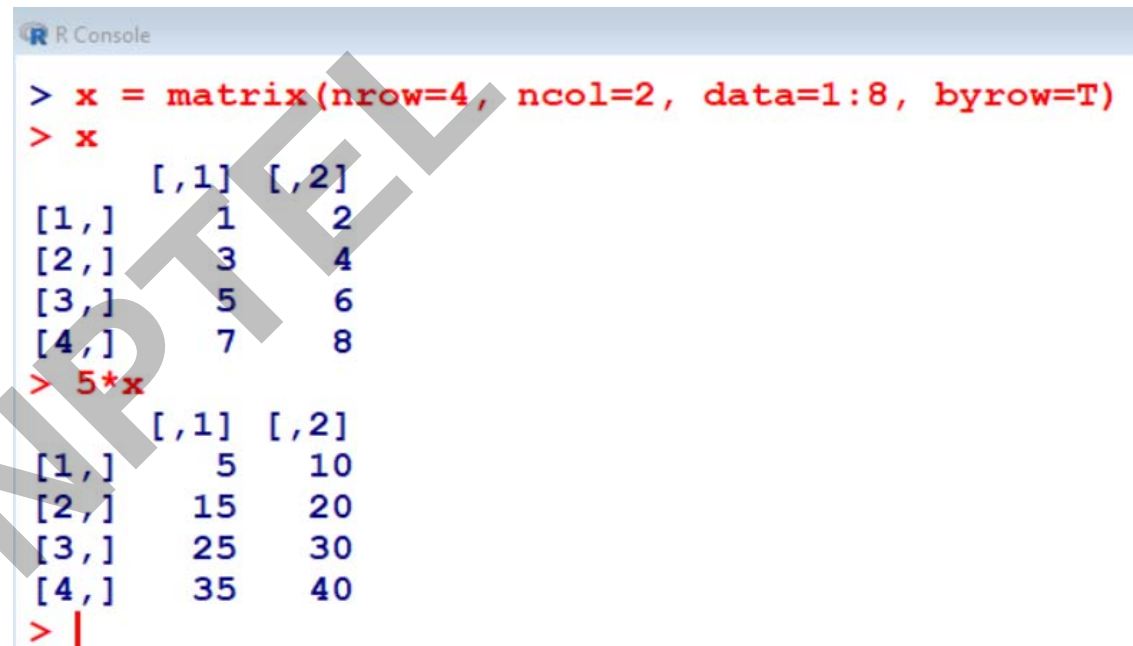
```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> 5*x
```

	[,1]	[,2]
[1,]	5	10
[2,]	15	20
[3,]	25	30
[4,]	35	40



```
R Console
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> 5*x
      [,1] [,2]
[1,]     5  10
[2,]    15  20
[3,]    25  30
[4,]    35  40
> |
```

Division of a matrix with a constant

Every element is divided by the scalar

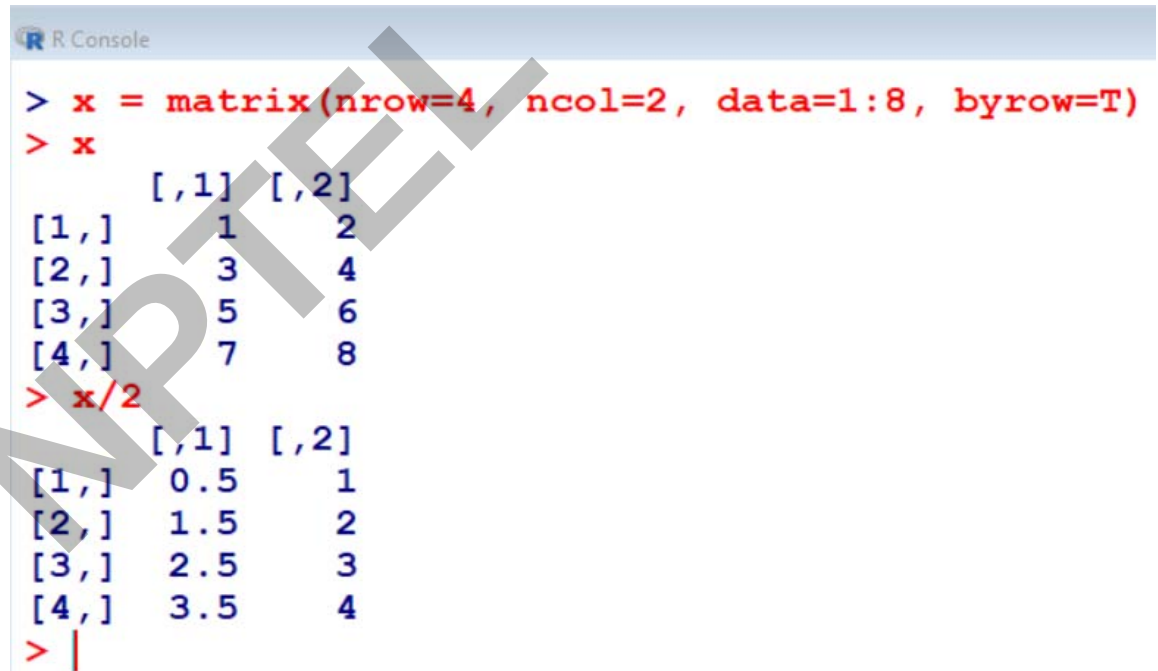
```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> x/2
```

	[,1]	[,2]
[1,]	0.5	1
[2,]	1.5	2
[3,]	2.5	3
[4,]	3.5	4



```
R Console
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> x/2
      [,1] [,2]
[1,]  0.5    1
[2,]  1.5    2
[3,]  2.5    3
[4,]  3.5    4
> |
```

Addition and subtraction of matrices

Addition and subtraction of matrices (of same dimensions) can be executed with the usual operators + and -

```
x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
y = matrix(nrow=4, ncol=2, data=11:18, byrow=T)
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> y
```

	[,1]	[,2]
[1,]	11	12
[2,]	13	14
[3,]	15	16
[4,]	17	18

Addition and subtraction of matrices

```
> x + y
```

	[,1]	[,2]
[1,]	12	14
[2,]	16	18
[3,]	20	22
[4,]	24	26

```
> x - y
```

	[,1]	[,2]
[1,]	-10	-10
[2,]	-10	-10
[3,]	-10	-10
[4,]	-10	-10

R Console

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> y = matrix(nrow=4, ncol=2, data=11:18, byrow=T)
> y
      [,1] [,2]
[1,]   11   12
[2,]   13   14
[3,]   15   16
[4,]   17   18
> x+y
      [,1] [,2]
[1,]   12   14
[2,]   16   18
[3,]   20   22
[4,]   24   26
> |
```

Addition and subtraction of matrices

```
> x - y
```

	[,1]	[,2]
[1,]	-10	-10
[2,]	-10	-10
[3,]	-10	-10
[4,]	-10	-10

R Console

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> y = matrix(nrow=4, ncol=2, data=11:18, byrow=T)
> y
      [,1] [,2]
[1,]   11   12
[2,]   13   14
[3,]   15   16
[4,]   17   18
> x-y
      [,1] [,2]
[1,]   -10  -10
[2,]   -10  -10
[3,]   -10  -10
[4,]   -10  -10
> |
```

Foundations of R Software

Lecture 15

Basics of Calculations

::::

Matrix Operations – Mathematical and Other Operations

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

Addition and subtraction of matrices

Example:

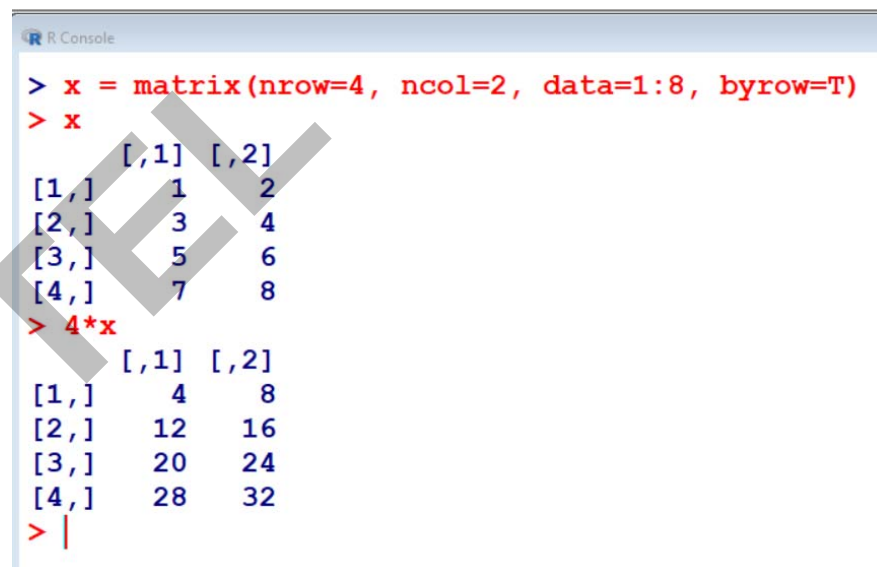
```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[,1]	[,2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> 4*x
```

	[,1]	[,2]
[1,]	4	8
[2,]	12	16
[3,]	20	24
[4,]	28	32



The screenshot shows an R console window with the following commands and output:

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> 4*x
      [,1] [,2]
[1,]     4    8
[2,]    12   16
[3,]    20   24
[4,]    28   32
> |
```

Addition and subtraction of matrices

Example:

```
> x + 4*x
```

	[,1]	[,2]
[1,]	5	10
[2,]	15	20
[3,]	25	30
[4,]	35	40

```
> 4*x - x
```

	[,1]	[,2]
[1,]	3	6
[2,]	9	12
[3,]	15	18
[4,]	21	24

```
R Console
> x
      [,1] [,2]
[1,]     1     2
[2,]     3     4
[3,]     5     6
[4,]     7     8
> 4*x
      [,1] [,2]
[1,]     4     8
[2,]    12    16
[3,]    20    24
[4,]    28    32
> x + 4*x
      [,1] [,2]
[1,]     5    10
[2,]    15    20
[3,]    25    30
[4,]    35    40
> 4*x - x
      [,1] [,2]
[1,]     3     6
[2,]     9    12
[3,]    15    18
[4,]    21    24
> |
```

Multiplication of matrices

Multiplication of matrices can be executed with the operator `%*%`

```
x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
y = matrix(nrow=2, ncol=4, data=11:18, byrow=T)
```

```
> x
```

	[, 1]	[, 2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> y
```

	[, 1]	[, 2]	[, 3]	[, 4]
[1,]	11	12	13	14
[2,]	15	16	17	18

Multiplication of matrices

> x%y**

	[, 1]	[, 2]	[, 3]	[, 4]
[1,]	41	44	47	50
[2,]	93	100	107	114
[3,]	145	156	167	178
[4,]	197	212	227	242

> y%x**

	[, 1]	[, 2]
[1,]	210	260
[2,]	274	340

```
R Console
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> y = matrix(nrow=2, ncol=4, data=11:18, byrow=T)
> x
      [,1] [,2]
[1,]     1     2
[2,]     3     4
[3,]     5     6
[4,]     7     8
> y
      [,1] [,2] [,3] [,4]
[1,]    11    12    13    14
[2,]    15    16    17    18
> x**%y
      [,1] [,2] [,3] [,4]
[1,]    41    44    47    50
[2,]    93   100   107   114
[3,]   145   156   167   178
[4,]   197   212   227   242
> y**%x
      [,1] [,2]
[1,]   210   260
[2,]   274   340
> |
```

Multiplication of matrices

Another example: Consider the multiplication of X' and X

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[, 1]	[, 2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> t(x)
```

	[, 1]	[, 2]	[, 3]	[, 4]
[1,]	1	3	5	7
[2,]	2	4	6	8

Multiplication of matrices

```
> t(x) %*% x
```

	[,1]	[,2]
[1,]	84	100
[2,]	100	120

```
> x %*% t(x)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	5	11	17	23
[2,]	11	25	39	53
[3,]	17	39	61	83
[4,]	23	53	83	113

```
R Console
> x := matrix(nrow=4, ncol=2, data=1:8, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
> t(x)
      [,1] [,2] [,3] [,4]
[1,]    1    3    5    7
[2,]    2    4    6    8
> t(x) %*% x
      [,1] [,2]
[1,]   84  100
[2,]  100  120
> x %*% t(x)
      [,1] [,2] [,3] [,4]
[1,]    5   11   17   23
[2,]   11   25   39   53
[3,]   17   39   61   83
[4,]   23   53   83  113
```

Multiplication of matrices

Cross Product of a matrix X , $X'X$, with `crossprod()`

```
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)
```

```
> x
```

	[, 1]	[, 2]
[1,]	1	2
[2,]	3	4
[3,]	5	6
[4,]	7	8

```
> t(x)
```

	[, 1]	[, 2]	[, 3]	[, 4]
[1,]	1	3	5	7
[2,]	2	4	6	8

Multiplication of matrices

```
> crossprod(x)
```

```
      [,1] [,2]  
[1,]    84  100  
[2,]   100  120
```

```
R Console  
  
> x = matrix(nrow=4, ncol=2, data=1:8, byrow=T)  
> x  
      [,1] [,2]  
[1,]    1    2  
[2,]    3    4  
[3,]    5    6  
[4,]    7    8  
  
> crossprod(x)  
      [,1] [,2]  
[1,]    84  100  
[2,]   100  120  
  
>  
  
> t(x) %*% x  
      [,1] [,2]  
[1,]    84  100  
[2,]   100  120  
  
> |
```

Note: Command **crossprod()** executes the multiplication faster than the conventional method with **t(x)%*%x**

Concatenating matrices

Concatenating matrices row wise: `rbind(x, y)`

Concatenating matrices column wise: `cbind(x, y)`

```
x = matrix(nrow=3, ncol=2, data=1:6, byrow=T)
```

```
y = matrix(nrow=3, ncol=2, data=11:16, byrow=T)
```

<pre>> x</pre>	<pre>> y</pre>	<pre>> rbind(x, y)</pre>
<pre> [,1] [,2]</pre>	<pre> [,1] [,2]</pre>	<pre> [,1] [,2]</pre>
<pre>[1,] 1 2</pre>	<pre>[1,] 11 12</pre>	<pre>[1,] 1 2</pre>
<pre>[2,] 3 4</pre>	<pre>[2,] 13 14</pre>	<pre>[2,] 3 4</pre>
<pre>[3,] 5 6</pre>	<pre>[3,] 15 16</pre>	<pre>[3,] 5 6</pre>
		<pre>[4,] 11 12</pre>
		<pre>[5,] 13 14</pre>
		<pre>[6,] 15 16</pre>

Concatenating matrices

Concatenating matrices row wise: `rbind(x, y)`

Concatenating matrices column wise: `cbind(x, y)`

```
x = matrix(nrow=3, ncol=2, data=1:6, byrow=T)
```

```
y = matrix(nrow=3, ncol=2, data=11:16, byrow=T)
```

<pre>> x</pre>			<pre>> y</pre>		
	<pre>[,1]</pre>	<pre>[,2]</pre>		<pre>[,1]</pre>	<pre>[,2]</pre>
<pre>[1,]</pre>	1	2	<pre>[1,]</pre>	11	12
<pre>[2,]</pre>	3	4	<pre>[2,]</pre>	13	14
<pre>[3,]</pre>	5	6	<pre>[3,]</pre>	15	16

<pre>> cbind(x, y)</pre>				
	<pre>[,1]</pre>	<pre>[,2]</pre>	<pre>[,3]</pre>	<pre>[,4]</pre>
<pre>[1,]</pre>	1	2	11	12
<pre>[2,]</pre>	3	4	13	14
<pre>[3,]</pre>	5	6	15	16

Concatenating matrices

Concatenating matrices row wise: `rbind(x, y)`

Concatenating matrices column wise: `cbind(x, y)`

```
R Console
> x = matrix(nrow=3, ncol=2, data=1:6, byrow=T)
> y = matrix(nrow=3, ncol=2, data=11:16, byrow=T)
> x
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
> y
      [,1] [,2]
[1,]   11   12
[2,]   13   14
[3,]   15   16
> rbind(x, y)
      [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]   11   12
[5,]   13   14
[6,]   15   16
> cbind(x, y)
      [,1] [,2] [,3] [,4]
[1,]    1    2   11   12
[2,]    3    4   13   14
[3,]    5    6   15   16
```

Inverse of matrix

`solve()` finds the inverse of a positive definite matrix

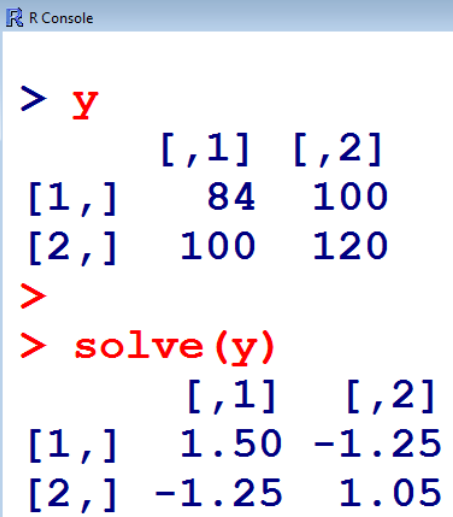
```
> y = matrix( nrow = 2, ncol = 2, byrow = T,  
data = c(84,100,100,120))
```

```
> y
```

```
      [,1] [,2]  
[1,]   84  100  
[2,]  100  120
```

```
> solve(y)
```

```
      [,1] [,2]  
[1,]  1.50 -1.25  
[2,] -1.25  1.05
```



```
R Console  
  
> y  
      [,1] [,2]  
[1,]   84  100  
[2,]  100  120  
>  
> solve(y)  
      [,1] [,2]  
[1,]  1.50 -1.25  
[2,] -1.25  1.05
```

Eigen values and eigen vectors of matrix

eigen() finds the eigen values and eigen vectors of a positive definite matrix

```
y = matrix( nrow = 2, ncol = 2, byrow = T, data  
= c(84,100,100,120))
```

```
> y
```

```
      [,1] [,2]  
[1,]   84  100  
[2,]  100  120
```

Eigen values and eigen vectors of matrix

```
> eigen(y)
```

```
$values
```

```
[1] 203.6070864    0.3929136
```

```
$vectors
```

```
          [,1]      [,2]  
[1, ] 0.6414230 -0.7671874  
[2, ] 0.7671874  0.6414230
```

```
R Console  
  
> y  
      [,1] [,2]  
[1,]   84  100  
[2,]  100  120  
> eigen(y)  
$values  
[1] 203.6070864    0.3929136  
  
$vectors  
          [,1]      [,2]  
[1,] 0.6414230 -0.7671874  
[2,] 0.7671874  0.6414230
```

Foundations of R Software

Lecture 16

Basics of Calculations

::::

Logical Operators

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

Logical Operators and Comparisons

The following table shows the operations and functions for logical comparisons (True or False).

Operator	Executions
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Exactly equal to
!=	Not equal to
!	Negation (not)
&, &&	and
,	or

Operator	Executions
<code>xor(x, y)</code>	either... or (exclusive)
<code>isTRUE(x)</code>	test if <code>x</code> is TRUE
<code>isFALSE(x)</code>	test if <code>x</code> is FALSE
TRUE	true
FALSE	false

Logical Operators and Comparisons

Operator	Executions
&, &&	and
,	or

- The shorter form performs element-wise comparisons in almost the same way as arithmetic operators.
- The longer form evaluates left to right examining only the first element of each vector. Evaluation proceeds only until the result is determined.

Examples: Use of | and ||

```
> x=8
```

```
> (x < 10) || (x < 2)
```

```
[1] TRUE
```

```
> x=18
```

```
> (x < 10) || (x < 2)
```

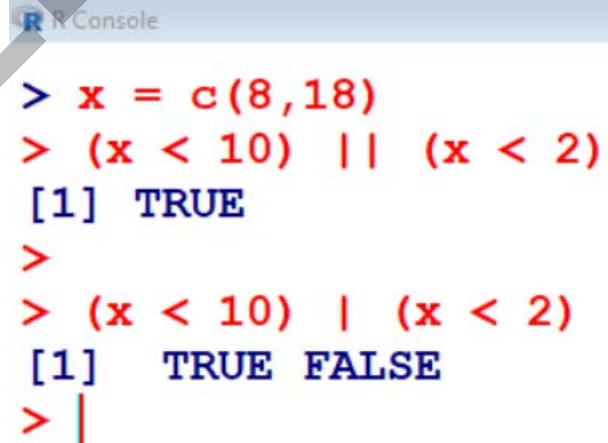
```
[1] FALSE
```

```
R Console  
  
> x=8  
> (x < 10) || (x < 2)  
[1] TRUE  
> x=18  
> (x < 10) || (x < 2)  
[1] FALSE  
>
```

Examples: Use of | and ||

```
> x = c(8,18)
> (x < 10) || (x < 2)
[1] TRUE
```

|| Operates only on the first element and not on remaining elements.

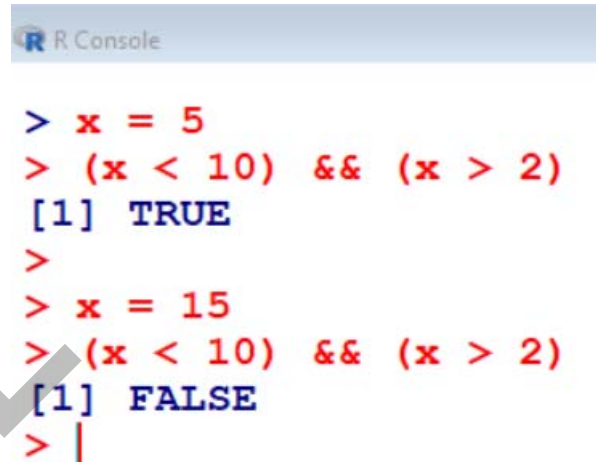


```
R Console
> x = c(8,18)
> (x < 10) || (x < 2)
[1] TRUE
>
> (x < 10) | (x < 2)
[1] TRUE FALSE
> |
```

```
> (x < 10) | (x < 2)
[1] TRUE FALSE
| Operates on all the elements.
```

Examples: Use of & and &&

```
> x = 5  
> (x < 10) && (x > 2)  
[1] TRUE
```

A screenshot of an R console window with a light blue header bar containing the R logo and the text "R Console". The console shows the following commands and output in red text:

```
> x = 5  
> (x < 10) && (x > 2)  
[1] TRUE  
>  
> x = 15  
> (x < 10) && (x > 2)  
[1] FALSE  
> |
```

```
> x = 15  
> (x < 10) && (x > 2)  
[1] FALSE
```

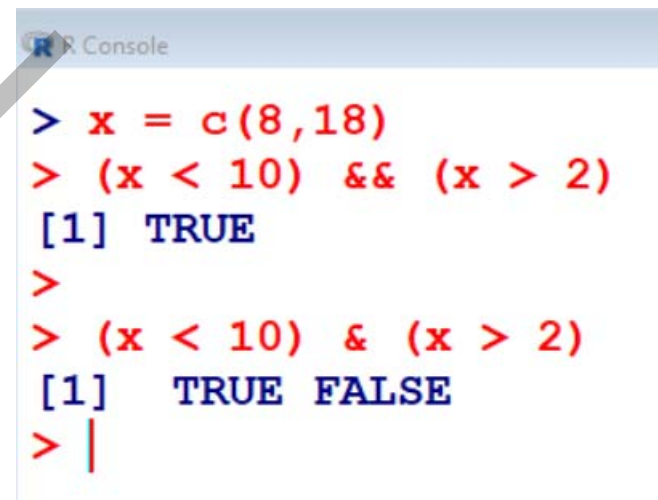
Examples: Use of & and &&

```
> x = c(8,18)
> (x < 10) && (x > 2)
[1] TRUE
```

&& Operates only on the first element and not on remaining elements.

```
> (x < 10) & (x > 2)
[1] TRUE FALSE
```

& Operates on all the elements.



```
R Console
> x = c(8,18)
> (x < 10) && (x > 2)
[1] TRUE
>
> (x < 10) & (x > 2)
[1] TRUE FALSE
> |
```

Examples:

```
> x = 1:6          # Generates x=1,2,3,4,5,6
```

```
> (x > 2) & (x < 5) # Checks whether the  
                      values are greater  
                      than 2 and less than 5
```

```
[1] FALSE FALSE TRUE TRUE FALSE FALSE
```

```
> x[(x > 2) & (x < 5)] # Finds which values  
                        are greater than 2 and  
                        smaller than 5
```

```
[1] 3 4
```

Examples:

```
R R Console
> x = 1:6
> x
[1] 1 2 3 4 5 6
>
> (x > 2) & (x < 5)
[1] FALSE FALSE  TRUE  TRUE FALSE FALSE
>
> x[(x > 2) & (x < 5)]
[1] 3 4
```

Examples:

```
> x = 1:6          # Generates x=1,2,3,4,5,6
```

```
> (x > 2) | (x < 5) # Checks whether the  
                      values are greater  
                      than 2 or less than 5
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE
```

```
> x[(x > 2) | (x < 5)] # Finds which values  
                        are greater than 2 or  
                        smaller than 5
```

```
[1] 1 2 3 4 5 6
```

Examples:

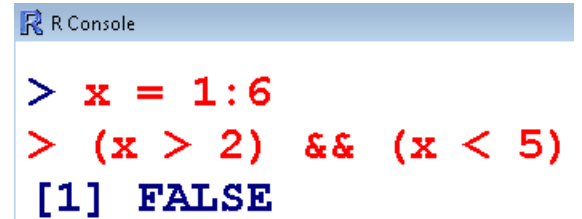
R Console

```
> x = 1:6  
> (x > 2) | (x < 5)  
[1] TRUE TRUE TRUE TRUE TRUE TRUE  
>  
> x[(x > 2) | (x < 5)]  
[1] 1 2 3 4 5 6
```

Example of “The longer form evaluates left to right examining only the first element of each vector”

```
> x = 1:6           # Generates x = 1,2,3,4,5,6
```

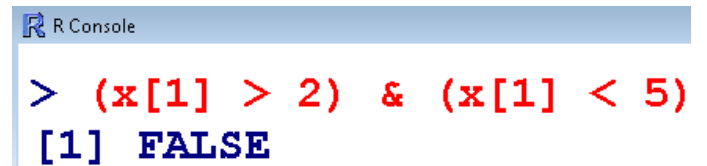
```
> (x > 2) && (x < 5)
[1] FALSE
```



```
R Console
> x = 1:6
> (x > 2) && (x < 5)
[1] FALSE
```

is equivalent to:

```
> (x[1] > 2) & (x[1] < 5)
[1] FALSE
```



```
R Console
> (x[1] > 2) & (x[1] < 5)
[1] FALSE
```

Note that `x[1]` is only the first element in `x`

Foundations of R Software

Lecture 17

Basics of Calculations

::::

Relational and Logical Operators

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

Relational Operators and Comparisons

The following table shows the operations and functions for relational comparisons (True or False).

TRUE and **FALSE** are reserved words denoting logical constants.

Operator	Executions
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Exactly equal to
!=	Not equal to
!	Negation (not)

Zero is considered as
FALSE
and
non-zero numbers are
taken as **TRUE**

Logical Operators and Comparisons

Operator	Executions	
!	Logical not	
&	Element-wise logical and	Performs operation element-wise
&&	Logical and	examines only the first element of the operands
 	Element-wise logical or	Performs operation element-wise
 	Logical or	examines only the first element of the operands

Logical Operators and Comparisons

Operator	Executions
xor(x, y)	Either x or y (x or y but not x and y)
isTRUE(x) isFALSE(x)	test if x is TRUE test if x is FALSE
TRUE FALSE	true false

Example of Standard logical operations

Truth table

Statement 1 :: (x)	Statement 2 :: (y)	Outcome :: x and y	Outcome :: x or y
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Example of Standard logical operations

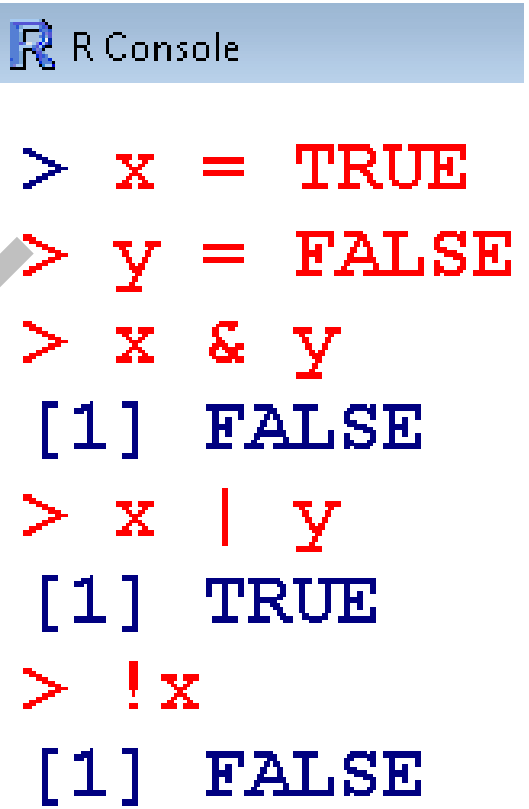
```
> x = TRUE
```

```
> y = FALSE
```

```
> x & y          # x AND y  
[1] FALSE
```

```
> x | y          # x OR y  
[1] TRUE
```

```
> !x             # negation of x  
[1] FALSE
```



```
R Console  
  
> x = TRUE  
> y = FALSE  
> x & y  
[1] FALSE  
> x | y  
[1] TRUE  
> !x  
[1] FALSE
```

Example

```
> x = 5
```

```
> Logical1 = (x > 2)
```

```
> Logical1
```

```
[1] TRUE
```

```
> is.logical(Logical1)
```

```
[1] TRUE
```

```
> Logical2 = (x < 10)
```

```
> Logical2
```

```
[1] TRUE
```

```
> is.logical(Logical2)
```

```
[1] TRUE
```

```
> Logical3 = (x != 5)
```

```
> Logical3
```

```
[1] FALSE
```

```
> is.logical(Logical3)
```

```
[1] TRUE
```

Example

```
R Console

> x = 5
> Logical1 = (x > 2)
> Logical1
[1] TRUE
> is.logical(Logical1)
[1] TRUE
>
> Logical2 = (x < 10)
> Logical2
[1] TRUE
> is.logical(Logical2)
[1] TRUE
>
> Logical3 = (x != 5)
> Logical3
[1] FALSE
> is.logical(Logical3)
[1] TRUE
> |
```

Example

```
> x = 5
```

```
> Logical4 = (2*x > 11)
```

```
> Logical4
```

```
[1] FALSE
```

```
> is.logical(Logical4)
```

```
[1] TRUE
```

```
> Logical5 = (3*x < 20)
```

```
> Logical5
```

```
[1] TRUE
```

```
> is.logical(Logical5)
```

```
[1] TRUE
```

R Console

```
> x = 5
> Logical4 = (2*x > 11)
> Logical4
[1] FALSE
> is.logical(Logical4)
[1] TRUE
>
> Logical5 = (3*x < 20)
> Logical5
[1] TRUE
> is.logical(Logical5)
[1] TRUE
> |
```

Examples:

```
> 8 > 7  
[1] TRUE
```

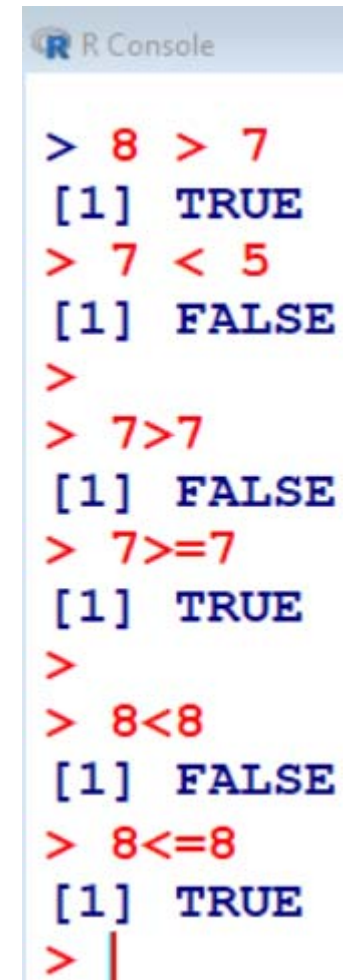
```
> 7 < 5  
[1] FALSE
```

```
> 7 > 7  
[1] FALSE
```

```
> 7 >= 7  
[1] TRUE
```

```
> 8 < 8  
[1] FALSE
```

```
> 8 <= 8  
[1] TRUE
```

A screenshot of an R console window showing a series of comparison operations and their results. The window title is 'R Console'. The text is as follows:

```
> 8 > 7  
[1] TRUE  
> 7 < 5  
[1] FALSE  
>  
> 7>7  
[1] FALSE  
> 7>=7  
[1] TRUE  
>  
> 8<8  
[1] FALSE  
> 8<=8  
[1] TRUE  
> |
```

Examples:

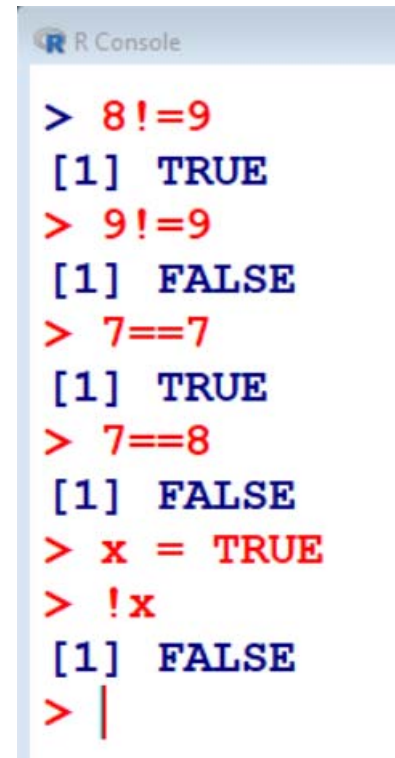
```
> 8 != 9  
[1] TRUE
```

```
> 9 != 9  
[1] FALSE
```

```
> 7 == 7  
[1] TRUE
```

```
> 7 == 8  
[1] FALSE
```

```
> x = TRUE  
> !x  
[1] FALSE
```

A screenshot of an R console window with a light blue header bar containing the R logo and the text "R Console". The console displays a series of R commands and their outputs. The commands are: 8 != 9, 9 != 9, 7 == 7, 7 == 8, x = TRUE, !x, and a final prompt line. The outputs are: [1] TRUE, [1] FALSE, [1] TRUE, [1] FALSE, [1] TRUE, [1] FALSE, and a vertical bar. The commands and their corresponding outputs are color-coded: commands are in red, and outputs are in blue.

```
R Console  
> 8 != 9  
[1] TRUE  
> 9 != 9  
[1] FALSE  
> 7 == 7  
[1] TRUE  
> 7 == 8  
[1] FALSE  
> x = TRUE  
> !x  
[1] FALSE  
> |
```

Examples:

```
> x = c(1, 2, 3)
```

```
> y = c(4, 5, 6)
```

```
> x > y      # Compares 1 > 4, 2 > 5, 3 > 6  
[1] FALSE FALSE FALSE
```

```
> x < y      # Compares 1 < 4, 2 < 5, 3 < 6  
[1] TRUE TRUE TRUE
```

```
> x != y     # Compares 1 ≠ 4, 2 ≠ 5, 3 ≠ 6  
[1] TRUE TRUE TRUE
```

```
> x == y     # Compares 1 = 4, 2 = 5, 3 = 6  
[1] FALSE FALSE FALSE
```

```
R Console  
  
> x = c(1, 2, 3)  
> y = c(4, 5, 6)  
> x > y  
[1] FALSE FALSE FALSE  
>  
> x < y  
[1] TRUE TRUE TRUE  
>  
> x != y  
[1] TRUE TRUE TRUE  
>  
> x == y  
[1] FALSE FALSE FALSE  
> |
```

Examples:

Is 8 less than 6?

```
> isTRUE(8 < 6)
[1] FALSE
```

Is 8 greater than 6?

```
> isTRUE(8 > 6)
[1] TRUE
```

Is 5 less than 8?

```
> isFALSE(5 < 8)
[1] FALSE
```

Is 5 greater than 8?

```
> isFALSE(5 > 8)
[1] TRUE
```

R Console

```
> isTRUE(8<6)
[1] FALSE
>
> isTRUE(8>6)
[1] TRUE
>
> isFALSE(5<8)
[1] FALSE
>
> isFALSE(5>8)
[1] TRUE
> |
```